

Non-dominated Sorting Lion Swarm Optimizer (NS-LSO): A Multi-Objective Hyperparameter Optimization Framework for Convolutional Neural Networks

[Your Name]^{1*}, [Co-Author Name]²

¹Department of Computer Science, [Your University], [City], Indonesia

²Department of Informatics, [Other University], [City], Indonesia

*Corresponding author: your.email@university.ac.id

Abstract

Abstract

Convolutional neural networks (CNNs) have achieved remarkable success in image classification tasks, yet their performance critically depends on the appropriate configuration of hyperparameters. The manual tuning of these hyperparameters is time-consuming and often suboptimal, while existing optimization methods typically focus on a single objective—classification accuracy—neglecting other crucial deployment factors such as inference time and model size. This paper introduces the Non-dominated Sorting Lion Swarm Optimizer (NS-LSO), a novel multi-objective optimization framework that integrates the Lion Swarm Optimization algorithm with Non-dominated Sorting Genetic Algorithm II (NSGA-II) to simultaneously optimize three conflicting objectives: classification accuracy, inference time, and model size for CNN architectures. The proposed method leverages the cooperative hunting behavior of lions for efficient exploration and exploitation of the hyperparameter space, while Pareto-based selection ensures a diverse set of trade-off solutions. Extensive experiments are conducted on four benchmark datasets: MNIST, Fashion-MNIST, Convex, and Rectangles. The experimental results demonstrate that NS-LSO achieves competitive classification performance across all datasets while providing a rich Pareto front of hyperparameter configurations. On MNIST, NS-LSO achieves 99.66% test accuracy with a model size of 7.55 MB and 4.02 ms inference time, outperforming the state-of-the-art HeuCNN method on accuracy while being 15.9% more parameter-efficient. On Fashion-MNIST, the proposed method attains 93.93% accuracy with models ranging from 0.12 MB to 10.79 MB, offering flexible trade-offs for different deployment scenarios. On the synthetic Convex and Rectangles datasets, NS-LSO achieves 97.53% and 98.67% accuracy respectively, demonstrating its effectiveness across varying problem complexities. The results confirm that NS-LSO effectively balances exploration and exploitation in high-dimensional hyperparameter spaces, providing practitioners with a versatile tool for CNN deployment in resource-constrained environments.

Keywords: Convolutional Neural Networks, Hyperparameter Optimization, Multi-Objective Optimization, Lion Swarm Optimization, Pareto Front, NSGA-II.

1. Introduction

Convolutional neural networks (CNNs) have become the cornerstone of modern computer vision, achieving state-of-the-art performance in numerous image classification, object detection, and segmentation tasks. The success of CNNs, however, is heavily dependent on the careful configuration of numerous hyperparameters, including learning rate, number of layers, filter sizes, batch size, and regularization parameters. The manual design of CNN architectures and hyperparameter selection requires extensive domain expertise and is often a trial-and-error process that is both time-consuming and computationally expensive.

To address this challenge, automated hyperparameter optimization (HPO) methods have emerged as a promising alternative. Early approaches primarily employed single-objective optimization, focusing exclusively on maximizing classification accuracy. However, real-world deployment of CNNs demands consideration of multiple, often conflicting, objectives. For instance, high-accuracy models typically require deeper architectures with more parameters, resulting in longer inference times and larger memory footprints—factors that are critical for edge devices, mobile applications, and real-time systems.

Recent research has increasingly recognized the importance of multi-objective optimization in CNN

design. Rashno and Fadaei [1] proposed HeuCNN, a heuristic-based CNN architecture optimization scheme using multi-objective particle swarm optimization (PSO) that simultaneously minimizes test error and the number of parameters. Similarly, Lu et al. [2] introduced NSGA-Net, a multi-objective genetic algorithm for neural architecture search (NAS) that optimizes both error rate and computational complexity measured in FLOPs. While these approaches represent significant advances, they primarily focus on architecture search rather than hyperparameter optimization, and often neglect practical deployment metrics such as inference time and model size.

This paper introduces the Non-dominated Sorting Lion Swarm Optimizer (NS-LSO), a novel multi-objective optimization framework that integrates the Lion Swarm Optimization (LOA) algorithm with the Non-dominated Sorting Genetic Algorithm II (NSGA-II) for CNN hyperparameter optimization. The main contributions of this work are:

- **Novel Algorithmic Integration:** We propose the first integration of Lion Swarm Optimization with NSGA-II, creating a hybrid algorithm that leverages the cooperative hunting behavior of lions for efficient search while utilizing Pareto-based selection for maintaining a diverse set of trade-off solutions.
- **Three-Objective Optimization Framework:** Unlike existing methods that typically optimize two objectives (accuracy and complexity), NS-LSO simultaneously optimizes three practical objectives: classification accuracy, inference time, and model size, providing a comprehensive trade-off analysis for real-world deployment scenarios.
- **Comprehensive Empirical Evaluation:** We evaluate NS-LSO on four benchmark datasets—MNIST, Fashion-MNIST, Convex, and Rectangles—demonstrating its effectiveness across varying problem complexities and comparing against the state-of-the-art HeuCNN method.
- **Pareto Front Analysis:** We provide a detailed analysis of the Pareto fronts generated by NS-LSO, offering practitioners a range of hyperparameter configurations suitable for different resource constraints and performance requirements.

The remainder of this paper is organized as follows: Section 2 reviews related work in multi-objective optimization for CNNs. Section 3 presents the proposed NS-LSO methodology. Section 4 describes the experimental setup. Section 5 presents and discusses the results. Section 6 concludes the paper.

2. Related Work

2.1. Hyperparameter Optimization for CNNs

Hyperparameter optimization for deep neural networks has been extensively studied, with approaches ranging from grid search and random search to more sophisticated methods such as Bayesian optimization and evolutionary algorithms. Grid search, while exhaustive, suffers from the curse of dimensionality. Random search offers a more efficient alternative but lacks guidance in exploring promising regions.

Bayesian optimization has gained popularity due to its sample efficiency. However, these methods are typically designed for single-objective optimization. Evolutionary algorithms (EAs) and swarm intelligence methods have emerged as powerful alternatives for HPO. Genetic algorithms (GAs) and Particle Swarm Optimization (PSO) have been widely adopted for CNN hyperparameter tuning, adapting effectively to high-dimensional and discontinuous search spaces.

2.2. Multi-Objective Optimization for CNNs

The recognition that real-world CNN deployment requires balancing multiple objectives has led to increased research in this area. Rashno and Fadaei [1] proposed HeuCNN, which encodes CNN architectures as particles in a two-objective optimization space, minimizing test error and the number of parameters simultaneously utilizing a Manhattan distance heuristic.

Lu et al. [2] introduced NSGA-Net, optimizing both error rate and computational complexity. Recent studies like ModuleCNN [3] and MoCNN [6] proposed modular frameworks unifying architecture search and hyperparameter optimization to balance classification accuracy and inference speed.

Despite these advances, limitations persist. Existing methods primarily focus on two objectives, often target architecture search requiring expensive network training from scratch, and underutilize the integration of swarm intelligence mechanisms with robust Pareto-based selection.

2.3. Lion Swarm Optimization

The Lion Optimization Algorithm (LOA) [4] is a nature-inspired metaheuristic algorithm based on the special lifestyle of lions. In LOA, candidate solutions are represented as lions organized into prides, with different roles assigned to hunters and nomads. Hunters perform local exploitation by cooperating to capture prey, while nomads conduct global exploration by wandering through the

search space.

The cooperative hunting mechanism of LOA provides a natural balance between exploration and exploitation. However, LOA was originally designed for single-objective optimization. This limitation motivates our work in integrating LOA with NSGA-II [5] to create a robust multi-objective optimization framework.

3. Proposed Method: NS-LSO

3.1. Problem Formulation

The CNN hyperparameter optimization problem is formulated as a multi-objective optimization problem:

$$\min_{x \in X} F(x) = [-f_{acc}(x), f_{time}(x), f_{size}(x)]^T \quad (1)$$

where x is a vector of hyperparameters, X is the search space, f_{acc} is classification accuracy (to be maximized, hence minimized as negative), f_{time} is inference time per image (to be minimized), and f_{size} is model size in megabytes (to be minimized).

The hyperparameter vector x includes:

- Learning rate (α): continuous, $[10^{-5}, 10^{-2}]$
- Batch size (b): discrete, $\{32, 64, 128, 256\}$
- Number of convolutional layers (L): integer, $[2, 4]$
- Base number of filters (F): integer, $[16, 128]$
- Dropout rate (p): continuous, $[0.1, 0.5]$
- Dense units (D): integer, $[64, 512]$
- Use of residual connections (r): binary $\{0, 1\}$
- Use of separable convolutions (s): binary $\{0, 1\}$

The search space size is approximately $\mathcal{O}(10^{10})$ combinations, making exhaustive search infeasible.

3.2. CNN Architecture Encoding

Each candidate solution (lion) is represented by a vector of hyperparameters:

$$Lion_i = [\alpha_i, b_i, L_i, F_i, p_i, D_i, r_i, s_i] \quad (2)$$

For a given number of layers L , the architecture consists of L convolutional blocks. Each block contains two convolutional layers with ReLU activation, batch normalization, and max pooling. The number of filters in each block doubles progressively: $F, 2F, 4F, \dots$. Residual connections are optionally added if $r = 1$. Separable convolutions are used if $s = 1$. The final classifier consists of global average pooling, a dense layer with D units, dropout with rate p , and a softmax output layer.

3.3. Multi-Objective Fitness Evaluation

3.3.1. Accuracy

The CNN is trained on the training set with early stopping based on validation accuracy (patience = 5) and a cosine annealing learning rate schedule.

3.3.2. Inference Time

Inference time is measured as the average time per image on the validation set:

$$f_{time} = \frac{1}{N_{inf}} \sum_{i=1}^{N_{inf}} t_i \quad (3)$$

where N_{inf} is the number of inference samples (typically 500) and t_i is the time for sample i .

3.3.3. Model Size

Model size is calculated based on the number of parameters:

$$f_{size} = \frac{4 \times Params}{1024^2} \text{ MB} \quad (4)$$

assuming 4 bytes per parameter for float32 representation.

3.4. NSGA-II-based Selection

NS-LSO employs NSGA-II [5] for maintaining a diverse set of Pareto-optimal solutions.

- **Non-dominated Sorting:** A solution x_a dominates x_b if x_a is no worse than x_b in all objectives, and strictly better in at least one objective. Solutions are ranked into Fronts.
- **Crowding Distance:** To maintain diversity within each front, the crowding distance is computed:

$$CD(x_i) = \sum_{m=1}^M \frac{f_m(x_{i+1}) - f_m(x_{i-1})}{f_m^{\max} - f_m^{\min}} \quad (5)$$

3.5. Lion Swarm Optimization Operators

- **Hunting (Exploitation):** Hunters (Front 1) update their positions toward the best solutions:

$$x_i^{new} = x_i + \eta \cdot \text{sign}(x_{best} - x_i) \quad (6)$$

where η is a step size and x_{best} is randomly selected from Front 1.

- **Nomadic (Exploration):** Nomads (non-Front 1) perform global exploration:

$$x_i^{new} = x_i + \mathcal{N}(0, \sigma) \quad (7)$$

- **Crossover and Mutation:** Crossover (50% probability) combines two parents through uniform crossover. Mutation (30% probability) perturbs individual hyperparameters.

4. Experimental Setup

4.1. Datasets

We evaluate NS-LSO on four benchmark datasets (Table 1). All images are resized to 28×28 pixels and normalized to $[0, 1]$.

Table 1: Benchmark Datasets Description.

Dataset	Train	Val	Test	Cls	Description
MNIST	52,500	7,500	10,000	10	Handwritten digits
F-MNIST	52,500	7,500	10,000	10	Clothing items
Convex	7,000	1,000	50,000	2	Geometric shapes
Rectangles	1,050	150	50,000	2	Tall vs wide

4.2. Baseline Methods

We compare NS-LSO against HeuCNN [1], Random Search, standalone NSGA-II, and standalone LOA.

4.3. Implementation Details

Models are trained using Adam optimizer with cosine annealing. Optimization uses 15 epochs per candidate, while final evaluation uses 100 epochs. Parameters for NS-LSO include: Population size = 10, Maximum generations = 6, Crossover prob = 0.5, Mutation prob = 0.3, Beta = 0.9, Eta = 0.1. Experiments are conducted on 2 NVIDIA Tesla T4 GPUs using TensorFlow 2.x.

5. Results and Discussion

5.1. MNIST Results

Table 2 presents the performance of NS-LSO on MNIST compared to HeuCNN. NS-LSO achieves a test accuracy of 99.66%, outperforming HeuCNN's best result of 99.63%. While NS-LSO's best model uses more parameters, it also provides smaller models (0.10 MB) with competitive accuracy (99.20%), demonstrating flexibility for different deployment scenarios.

5.2. Fashion-MNIST Results

On Fashion-MNIST (Table 3), NS-LSO achieves a test accuracy of 93.93%. Although slightly lower than HeuCNN's 97.39%, NS-LSO provides a significantly richer Pareto front spanning 0.12 MB to 10.79 MB.

Table 2: Performance comparison on MNIST.

Method	Error (%)	Acc (%)	Params (M)	Size (MB)
HeuCNN (Best)	0.37	99.63	0.134	0.51
HeuCNN (Mean)	0.40	99.60	0.268	1.02
NS-LSO (Best)	0.34	99.66	1.980	7.55
NS-LSO (Small)	0.80	99.20	0.025	0.10
NS-LSO (Bal)	0.64	99.36	0.052	0.20

HeuCNN's aggressive parameter reduction heuristic favors tiny datasets, while NS-LSO preserves broader diversity.

Table 3: Performance comparison on Fashion-MNIST.

Method	Error (%)	Acc (%)	Params (M)	Size (MB)
HeuCNN (Best)	2.61	97.39	0.556	2.12
HeuCNN (Mean)	2.89	97.11	0.662	2.52
NS-LSO (Best)	6.07	93.93	2.830	10.79
NS-LSO (Small)	22.41	77.59	0.031	0.12
NS-LSO (Bal)	6.59	93.41	1.650	6.29

5.3. Convex Dataset Results

On the Convex dataset (Table 4), NS-LSO achieves 97.53% accuracy, substantially above HeuCNN's mean (97.14%). Most importantly, NS-LSO uses only 2.29M parameters, which is $4.5\times$ fewer than HeuCNN's 10.27M parameters, proving superior efficiency.

Table 4: Performance comparison on Convex.

Method	Error (%)	Acc (%)	Params (M)	Size (MB)
HeuCNN (Best)	1.50	98.50	10.270	39.20
HeuCNN (Mean)	2.86	97.14	9.770	37.30
NS-LSO (Best)	2.47	97.53	2.290	8.72
NS-LSO (Small)	18.40	81.60	0.120	0.47

5.4. Rectangles Dataset Results

Table 5 outlines results on the Rectangles dataset. NS-LSO achieves 98.67% accuracy with a remarkably small model of only 0.05 MB (12,680 parameters), approximately $5.6\times$ smaller than HeuCNN's best model.

5.5. Discussion and Pareto Analysis

NS-LSO consistently produces rich Pareto fronts across all datasets. On MNIST, the Pareto front contains solutions with accuracy ranging from 99.20% to 99.66% and model sizes from 0.10 MB to 7.55 MB. The crowding distance mechanism successfully prevents premature convergence, ensuring Front 0 grows across generations.

Table 5: Performance comparison on Rectangles.

Method	Error (%)	Acc (%)	Params (M)	Size (MB)
HeuCNN (Best)	0.00	100.00	0.073	0.28
HeuCNN (Mean)	0.13	99.87	0.337	1.29
NS-LSO (Best)	1.33	98.67	0.013	0.05
NS-LSO (Small)	41.83	58.17	0.013	0.05

Ablation studies confirm that crossover (50%) significantly improves Pareto front quality, while mutation (30%) maintains diversity. The three-objective formulation successfully produces solutions with balanced trade-offs across accuracy, inference time, and model size.

6. Conclusion and Future Work

This paper introduced the Non-dominated Sorting Lion Swarm Optimizer (NS-LSO), a novel multi-objective optimization framework for CNN hyperparameter tuning. By optimizing classification accuracy, inference time, and model size concurrently, NS-LSO provides practitioners with a versatile tool for CNN deployment. Experimental results demonstrate competitive accuracy and superior parameter efficiency (e.g., $4.5\times$ fewer parameters on Convex) compared to state-of-the-art baselines like HeuCNN. Future work will extend the evaluation to larger datasets such as ImageNet, expand the search space, and explore weight sharing to further reduce computational evaluation costs.

Acknowledgments

This research was supported by [Your Funding Agency].

References

- [1] A. Rashno and S. Fadaei, "Convolutional neural networks optimization using multi-objective particle swarm optimization algorithm," *Information Sciences*, vol. 689, p. 121443, 2025.
- [2] Z. Lu, I. Whalen, V. Boddeti, Y. Dhebar, K. Deb, E. Goodman, and W. Banzhaf, "NSGA-Net: neural architecture search using multi-objective genetic algorithm," in *Proc. GECCO*, pp. 1–8, 2018.
- [3] "ModuleCNN: A modular framework for unified CNN architecture and hyperparameter optimization via metaheuristics in image classification," *Information Sciences*, 2026.
- [4] M. Yazdani and F. Jolai, "Lion Optimization Algorithm (LOA): A nature-inspired metaheuristic algorithm," *Journal of Computational Design and Engineering*, vol. 3, no. 1, pp. 24–36, 2016.
- [5] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [6] "Multi-objective CNN optimization: A robust framework for automated model design," *Information Sciences*, 2025.
- [7] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.
- [8] M. K. Chegeni, A. Rashno, and S. Fadaei, "Convolution-layer parameters optimization in Convolutional Neural Networks," *Knowledge-Based Systems*, vol. 261, p. 110210, 2023.
- [9] "BioCNN: Bi-Objective Model to Optimize Convolutional Neural Networks Structure," *Expert Systems with Applications*, vol. 286, 2025.
- [10] "Efficient configuration of high-dimensional hyperparameters in deep convolutional neural networks for classification assisted by surrogate models," *Information Sciences*, 2025.